

Fast & Resource-Efficient Toxic-Text Classifiers

Bilingual (RU + EN) safety filters: quality, speed, robustness

Arthur Babkin · Alexander Malyy

Generative AI · Spring 2026

Problem

Toxic-text filters sit on latency-sensitive paths. Three axes in tension:

Quality

Catch subtle, context-dependent toxicity. Miss rate directly harms users.

Speed

Sub-millisecond latency for real-time filtering. CPU-only budgets.

Robustness

Hold up against l33tspeak, character spacing, Cyrillic↔Latin homoglyphs.

Question: how do classical, full fine-tune, and LoRA compare on the same bilingual corpus?

Data

460K

samples (after LSH dedup)

4

public datasets merged

92K

test set (stratified, seed=42)

2

languages (RU + EN)

Russian: 55.7%

English: 44.3%

Safe: 80.1%

Toxic: 19.9%

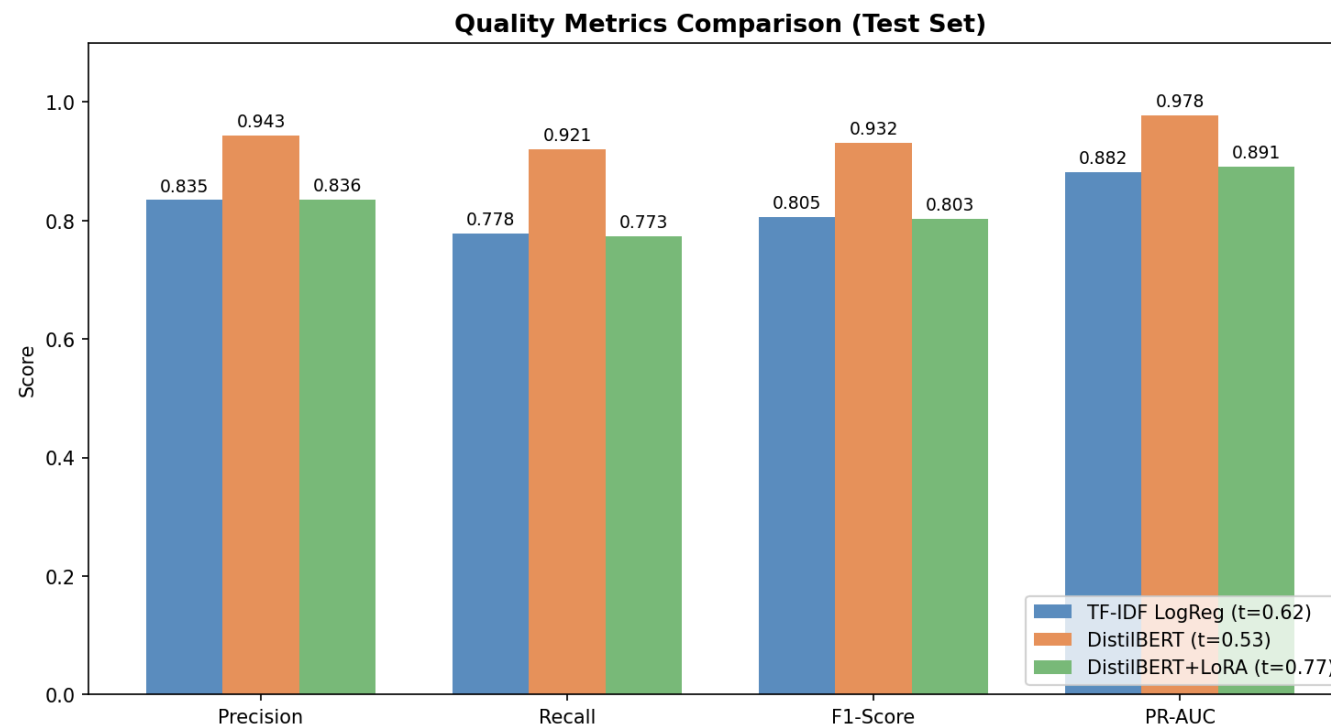
Cleaning stripped HTML/URLs/IPs/Wikipedia metadata. MinHash-LSH ($t=0.80$) removed 2.1% near-duplicates. Obfuscated stress set built for robustness eval.

Approach: three architectures, one protocol

Model	Params	Training data	Notes
TF-IDF + LogReg	10K features	133K (balanced)	unigrams + bigrams, L2
DistilBERT (full FT)	67.6M	331K	<code>distilbert-base-uncased</code> , 3 ep
DistilBERT + LoRA	739K (1.09%)	90K (balanced)	$r=8$, $\alpha=16$, $q_{lin} + v_{lin}$

Unified eval: same 92K test split. Thresholds tuned on val (sweep 0.10–0.90, argmax F1).

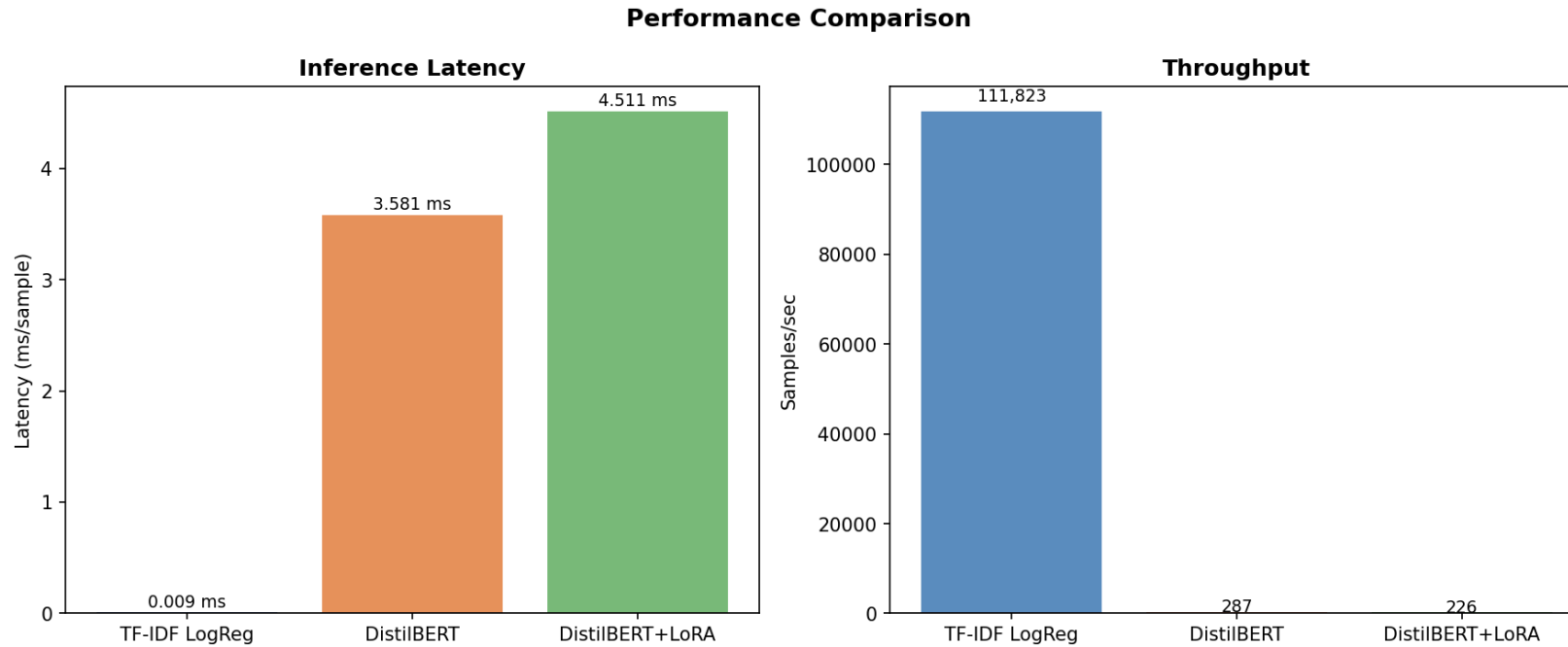
Headline results



Model	F1	Latency	Throughput
TF-IDF + LogReg	0.8054	0.009 ms	112K/s
DistilBERT	0.9318	3.58 ms	287/s
DistilBERT + LoRA	0.8035	4.51 ms	226/s

DistilBERT **+12.6pp** F1 over LogReg. LogReg **~390×** throughput of transformers.

Speed vs quality tradeoff



LoRA stuck in the **worst quadrant**: transformer latency, LogReg-level quality. Training efficiency only.

Per-language: Russian vs English

Model	EN F1	RU F1	Gap
TF-IDF + LogReg	0.8278	0.7851	+4.3pp
DistilBERT (full FT)	0.9364	0.9278	+0.9pp
DistilBERT + LoRA	0.8491	0.7608	+8.8pp

`distilbert-base-uncased` is English-pretrained, lowercases, strips accents. Cyrillic → byte-level WordPiece fallback.

Full fine-tuning absorbs the mismatch. LoRA cannot. 1% trainable params insufficient to realign Russian representation.

Ablations: overview

Four ablation studies, all evaluated under the same protocol as the main results:

#	Question	Models	Headline
1	Does balancing classes help?	LogReg / DistilBERT / LoRA	model-dependent
2	Does class-weighted loss help DistilBERT?	DistilBERT	−3.8pp F1
3	Is the LoRA gap from data size or architecture?	LoRA vs full FT @ 90K	+7.1pp full FT
4	Robustness under obfuscation + deobfuscation defense	all 3	76% recovery

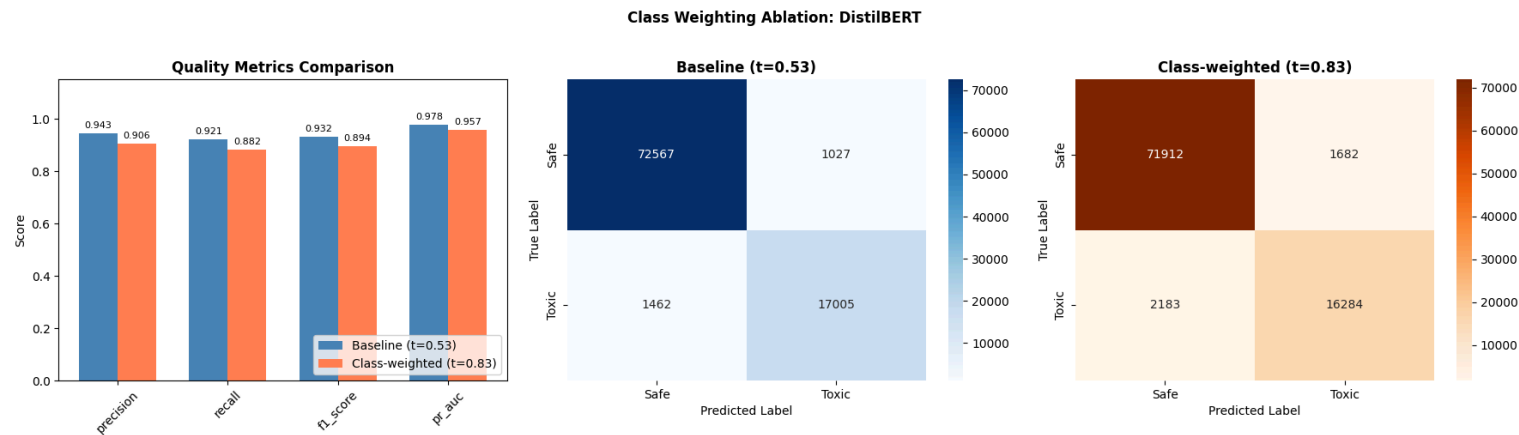
Each ablation derives modified training data from the global split. Val/test held fixed. Promoted weights: balanced LogReg (133K), balanced same-size LoRA (90K).

Ablation 1: class imbalance (undersampling)

Model	Baseline F1	Balanced F1	Δ	Verdict
LogReg (331K $\rightarrow$ 133K balanced)	0.7647	0.8054	+4.1pp	helps · promoted
DistilBERT (331K $\rightarrow$ 133K balanced)	0.9318	0.8844	−4.7pp	hurts (data loss)
LoRA (90K $\rightarrow$ 36K balanced, naive)	0.7929	0.7300	−6.3pp	hurts (data loss)
LoRA (90K $\rightarrow$ 90K balanced , same-size)	0.7929	0.8035	+1.1pp	helps · promoted

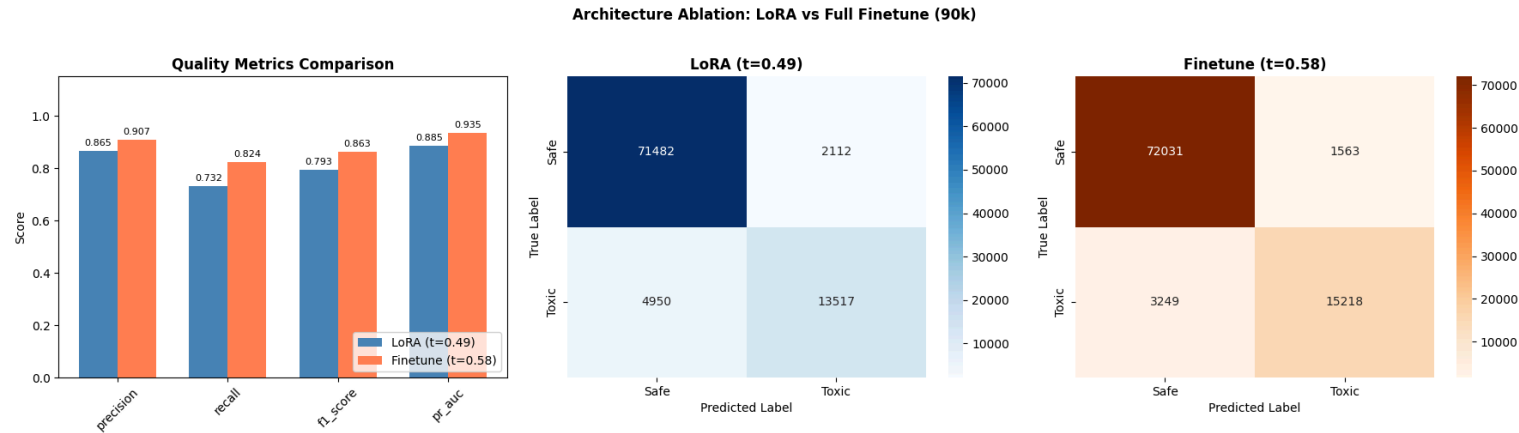
Pattern is model-dependent. Linear models gain from balanced priors. Pretrained transformers on 331K already have no majority-class bias to correct $\rightarrow$ undersampling only loses data. Holding LoRA's training size constant isolates the distribution effect from data loss.

Ablation 2: class weighting hurts full FT



DistilBERT @ 331K with 4× weight on toxic class: F1 **−3.8pp** (0.9318 → 0.8939) · PR-AUC **−2.1pp** · threshold drifts **0.53 → 0.83**. No majority-class bias to correct → gradient distortion breaks calibration. Both undersampling (−4.7pp) and weighting (−3.8pp) assume imbalance hurts; for full FT on 331K that assumption is wrong.

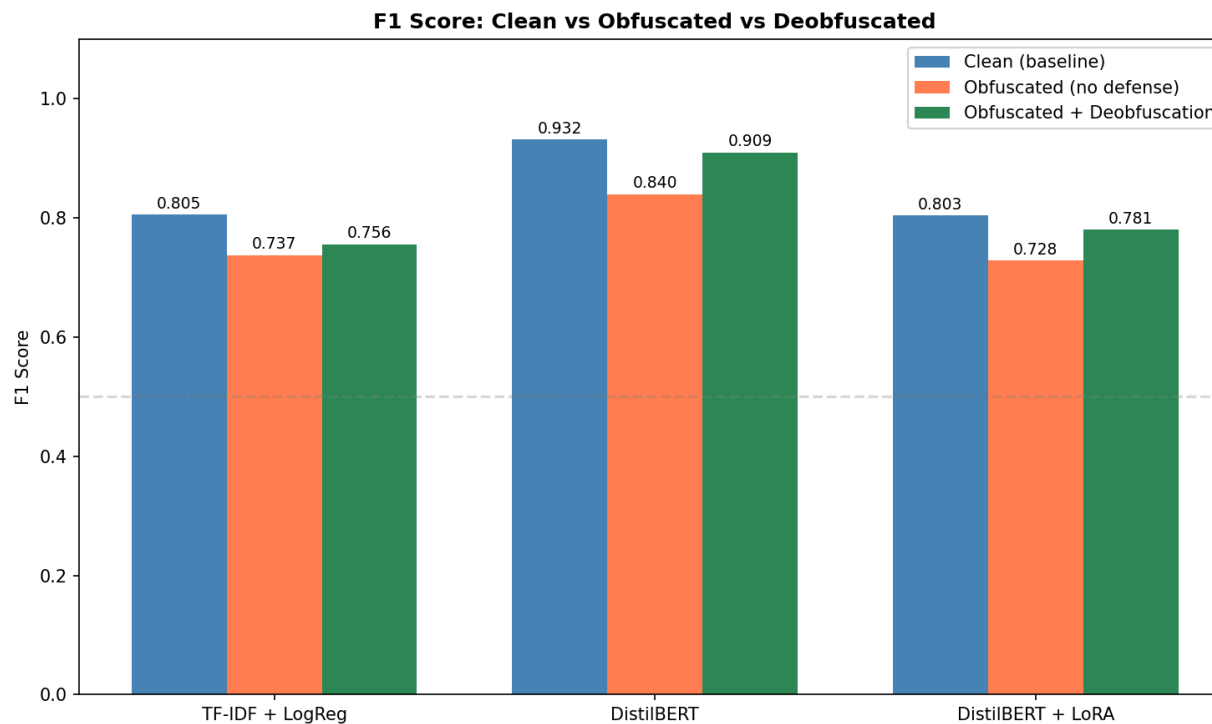
Ablation 3: LoRA vs full FT on same 90K



Model	Precision	Recall	F1	PR-AUC
LoRA (90K)	0.8649	0.7320	0.7929	0.8853
Full FT (90K)	0.9069	0.8241	0.8635	0.9354
Δ	+0.042	+0.092	+7.1pp	+0.050

Total LoRA vs full FT gap on the main split is **13.9pp**. Decomposes: **~7pp architecture**, **~7pp data size**. LoRA's 1% trainable budget limits discriminative features for the toxic class. Recall hit hardest (+9.2pp).

Ablation 4: robustness under obfuscation



Model	Clean	Obfuscated	+ Deobf.	Recovery
LogReg	0.8054	0.7373	0.7555	26.8%
DistilBERT	0.9318	0.8395	0.9095	75.8%
LoRA	0.8035	0.7283	0.7806	69.5%

Rule-based deobfuscation preprocessor: **zero retraining, ~76% recovery on transformers.**

Deployment recommendations

Use case	Pick	Why
Real-time, CPU-only	TF-IDF + LogReg	0.009 ms, 0.5 MB, F1=0.81
Batch safety review	DistilBERT + deobf.	F1=0.93 clean / 0.91 obf; smallest EN/RU gap
Rapid prototyping	LoRA	6× faster training, not for prod serving

Every transformer deployment → add the deobfuscation preprocessor. Free, effective.

Limitations

- **Domain mismatch.** Social-media comments only. LLM-output moderation unverified.
- **English-centric backbone.** XLM-R / mDeBERTa would avoid the EN/RU gap by design.
- **Single-seed training.** No variance across training seeds; small deltas indicative only.
- **Adversarial scope.** 4 rule-based obfuscation families. No paraphrase or semantic attacks.
- **Binary label collapse.** Severity, target, context discarded across 4 source datasets.
- **Rule-based deobfuscation.** Brittle to novel evasion; learned normalizer is future work.

Takeaways

1. Full fine-tuning dominates on quality (F1 0.93) *and* closes the RU/EN gap.
2. LoRA isn't a drop-in substitute: worst-quadrant latency/quality, largest language gap.
3. A free preprocessor recovers 76% of obfuscation loss on transformers.

Repo: `github.com/ArthurBabkin/GenAI-Safety-Fliter` · full report in `docs/final/final.md`

Team contributions

Arthur Babkin

- Collected data, prepared dataset
- Baseline TF-IDF implementation
- Ablation 2 (class weighting)
- Ablation 3 (LoRA vs full FT @ 90K)

Alexander Malyy

- Baseline DistilBERT + LoRA implementation
- Code refactor, unified methodology
- Ablation 1 (class imbalance)
- Ablation 4 (robustness + deobfuscation)